

API Monitoring — Hybrid Storage

Backend Observability & Chaos Engineering

Each [SLIDE] is one slide; [NOTES] are speaker notes

COVER SLIDE

API Infrastructure Monitoring
Anomaly Detection & Troubleshooting

Hybrid Storage Architecture: PostgreSQL + Redis

Monitoring Stack: Prometheus + Grafana

Chaos Engineering: Real-World Cascade Failure Simulation

CHAPTER 1

Architecture Decision & Design

[SLIDE 1-1] Why Hybrid Storage?

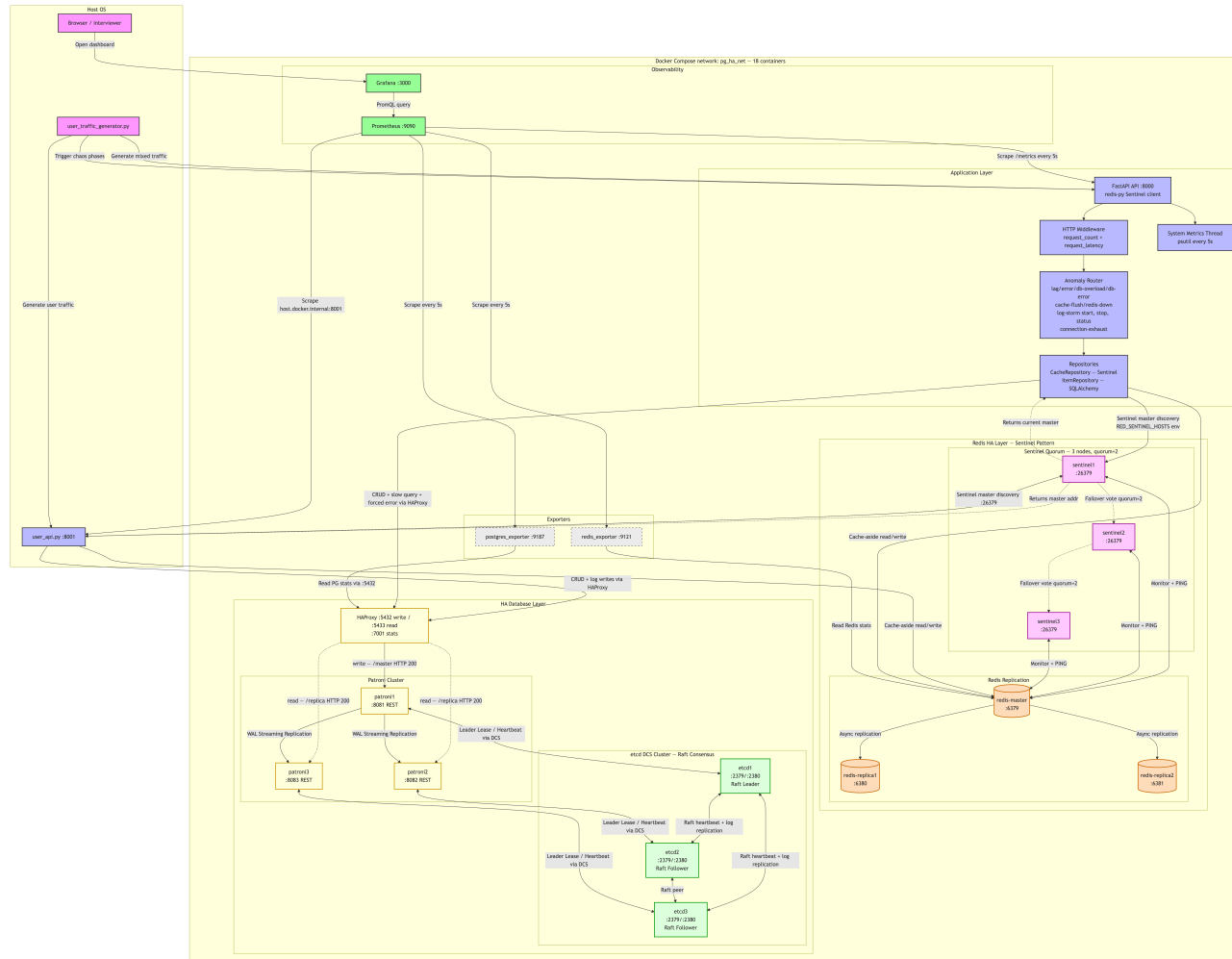
Core problem: the limits of a single database

High read frequency + a single PostgreSQL
↓
Every request hits the DB → connection-pool contention → rising latency

Solution: the Cache-Aside Pattern

Requirement	Choice	Reason
High-frequency, low-latency reads	Redis	In-memory access < 1ms, naturally suited to hot data
Persistence, consistency	PostgreSQL	ACID guarantees, data survives restarts
Graceful degradation	Two-tier architecture	When Redis is down, automatically fall back to the DB; the system does not crash

[SLIDE 1-2] System Architecture (Dual HA — 18 containers)



[SLIDE 1-2B] Data Flow (key points)

Data flow:

1. Online traffic enters user_api (8001) / main app (8000), first hitting the Redis Sentinel master (reducing DB pressure)
2. On a cache miss, query PostgreSQL (routed via HAProxy to the Patroni Primary) and backfill Redis
3. Metrics are scraped by Prometheus every 5 seconds; Grafana provides diagnostic visualization
4. Anomaly scenarios are actively triggered by user_traffic_generator.py --interactive (6 phases)
5. PostgreSQL HA failover is handled automatically by Patroni + etcd **Raft** ($RTO \leq 5$ seconds)
6. Redis HA failover is handled by Sentinel quorum=2 voting (down-after=5s, failover-timeout=10s)

[SLIDE 1-3] Deployment: Dual HA Infrastructure

PostgreSQL High Availability — Patroni + etcd Raft + HAProxy

```
etcd cluster (3 nodes – Raft Consensus) —————  
etcd1 :2379  etcd2 :2379  etcd3 :2379  
Raft: Leader Election + Log Replication + Quorum Write
```

↕ REST API (Patroni Leader Lease)

```
Patroni PostgreSQL cluster (3 nodes) —————  
patroni1 (replica) ← WAL streaming → patroni3 (primary) ★  
patroni2 (replica) ← WAL streaming → patroni3 (primary) ★
```

↕ health check /master /replica

```
HAProxy —————  
:5432 → primary (write)    :5433 → replicas (read)  
:7001/stats → real-time routing status
```

Redis High Availability — Sentinel Pattern

```
Redis Replication —————  
redis-master :6379  →  redis-replica1 :6380  
redis-master :6379  →  redis-replica2 :6381
```

↕ Monitor + PING (down-after=5s)

```
Sentinel Quorum (3 nodes, quorum=2) —————  
sentinel1/2/3 :26379 – failover-timeout=10s  
Failover: quorum=2 vote → Replica promoted to new Master
```

[SLIDE 1-3B] Deployment: Multi-Service Orchestration & Health Checks

Docker Compose multi-service orchestration (18 container services)

```
# HA database tier
etcd1/2/3      → Raft DCS, healthcheck: etcdctl member list
patroni1/2/3   → PostgreSQL HA node, healthcheck: GET /patroni
haproxy        → DB routing, healthcheck: stats page
# HA Redis tier
redis-master   → Redis master :6379
redis-replica1/2 → Redis replicas :6380/:6381
sentinel1/2/3  → Sentinel monitors :26379
# Application tier
api            → FastAPI :8000, depends_on: haproxy + sentinel1~3
# Cache & monitoring
redis_exporter, postgres_exporter, prometheus, grafana
```

Health-check design rationale:

Patroni uses the `/patroni` REST endpoint (not `/health`, which returns 503 during replica bootstrap).
HAProxy checks the Patroni REST API `/master` / `/replica` every 3 seconds and switches routing automatically.
Sentinel uses `nslookup` IP resolution at startup to avoid Alpine DNS latency issues.

[SLIDE 1-3C] Code Layering (OOP, 5 layers)

Code OOP layering (5-layer architecture)

<code>config.py</code>	→ Immutable settings (frozen dataclass)
<code>metrics.py</code>	→ All Prometheus metrics (Singleton pattern)
<code>repositories/</code>	→ Encapsulate all DB/Cache operations (Repository Pattern)
<code>routers/</code>	→ Pure business logic, accessing Repositories via DI
<code>main.py</code>	→ App Factory, assembly only, no business logic

CHAPTER 2

Monitoring & Observability

[SLIDE 2-1] Metric Selection Principles (this project)

Selection principles

- Map directly to failure modes: slow, errors, congestion, resource pressure
- Metrics are traceable to a tier: HTTP → Cache → DB → System
- Actionable: an abnormal metric maps to a concrete remediation action

Core metrics (RED + Resource)

Metric type	Metric name	Description
Rate	<code>http_requests_total</code>	Requests per second
Errors	<code>http_requests_total{http_status="500"}</code>	Error rate
Duration	<code>http_request_duration_seconds</code>	Request latency trend
Resource	<code>system_memory_usage_percent</code>	Reflects backlog and resource risk

[SLIDE 2-2] Metric Design: a Four-Tier Observation Chain

HTTP tier	Cache tier (Redis)	Database tier (PostgreSQL)	System resource tier (Container)
request_count → request_latency 5xx error rate	cache_hits_total → cache_misses_total redis_errors_total	db_query_duration → db_errors_total log_write_duration log_writes_total log_storm_active db_active_connections_held	system_cpu_usage_percent system_memory_usage_percent system_memory_used_bytes

Why monitor all four tiers?

Case: after a log storm starts, HTTP P99 keeps rising

- HTTP tier only: you know "the system is slow" but not who caused it
- Cache tier: Hit Rate is normal, ruling out a cache-invalidation avalanche
- DB tier: log_write_duration P99 and log_writes_total rise, confirming synchronous log writes are consuming DB resources
- System resource tier: system_memory_usage_percent climbs slowly, indicating request backlog and resource pressure are building

The four-tier observation chain lets you converge from "the system is slow" to "which tier, which resource, which behavior" is causing the anomaly.

[SLIDE 2-3] Grafana Dashboard Design Strategy

Row structure (5 thematic groups)

▼

 HTTP Traffic

Request Rate (RPS) | Latency P99/P95/P50 | 5xx Error Rate | Total Requests

▼

 Redis Cache

Cache Hit Rate (Gauge) | Hits vs Misses | Redis Errors

▼

 PostgreSQL

DB Query Duration P99 | DB Errors

▼

 Log Storm / Chaos Indicators

Log Storm Active (Gauge 0/1) | Log Write Rate | Log Write Duration P99 | DB Active Connections

▼

 System Resources

CPU Usage (Gauge) | Memory % (Gauge) | Memory Used (MB)

Key design choices

Panel	Visualization	Reason
Cache Hit Rate	Gauge	Intuitively shows health, green/yellow/red at a glance
Latency	Time Series	The trend matters more than a single number
Total Requests	Stat	Cumulative total, quickly confirms traffic exists

[SLIDE 2-4] System Resource Monitoring: the Container View

psutil + Gauge: 3 new metrics

```
# A background daemon thread samples once every 5 seconds
system_cpu_usage_percent    → psutil.cpu_percent()
system_memory_usage_percent → psutil.virtual_memory().percent
system_memory_used_bytes    → psutil.virtual_memory().used
```

Why do we need container-level CPU/Memory?

External exporters (redis_exporter, pg_exporter) only see their own services.

The API's own resource consumption is only visible inside the API container.

When a log storm causes many synchronous DB writes, the slow climb in memory is an early warning signal.

Threshold logic:

- Memory: yellow 70% (gives you time to investigate) → red 85% (must act immediately)
- CPU: yellow 70% → red 90% (a brief CPU spike is acceptable; sustained is dangerous)

CHAPTER 3

Chaos Engineering & Fault Diagnosis

[SLIDE 3-1] Typical Scenario: Core Query Service Connection-Pool Exhaustion

Incident background

Scenario: a high-traffic period after a new feature went live

Impact: many users could not access the core query service normally

Symptoms: API responses gradually slow → time out → the system becomes completely unresponsive

Root cause (post-incident analysis)

Step 1: the dev team shipped a "record API request logs to the DB" feature

Step 2: traffic in dev/test was low, so the problem went unnoticed

Step 3: under high production traffic, each request = 1 business DB query + 1 log INSERT

Step 4: the DB connection pool was exhausted (pool_size too small)

Step 5: requests began waiting for connections → latency rose

Step 6: requests piled up → more connection contention → vicious cycle

Step 7: the system crashed

Two pain points

1. **No monitoring:** the problem "slowly worsened" with no metric or alert
2. **Logs wiped out:** after a restart, the key error logs were gone and the problem could not be reconstructed

[SLIDE 3-2] Chaos Engineering Design: 7 Anomaly Endpoints

```
/api/anomaly/  
├── lag           → sleep 2.5s (generic latency)  
├── error         → HTTP 500 (application error)  
├── db-overload   → pg_sleep(3) (DB slow query)  
├── db-error      → query a non-existent table (forced DB error)  
├── cache-flush   → Redis FLUSHDB (Cache Stampede)  
├── redis-down    → connect to a non-existent Redis (disconnect simulation)  
├── log-storm/start → start synchronous DB log writes (core case)  
├── log-storm/stop → stop immediately, system recovers (Feature Flag pattern)  
└── connection-exhaust → hold 4/5 DB connections for 10s, then auto-release
```

Design principle: every endpoint has a corresponding observable metric

Chaos engineering is not about destruction; it is about making problems "visible within a controlled scope."

[SLIDE 3-3] Interactive Chaos Demo: 7 Phases (full run on 2026-06-02)

The full test sequence of chaos_scenario.py

```
Phase 1: Normal baseline      → API P50=7.8ms,    P99=32.1ms ← healthy level after Redis cache warm-up
Phase 2: Log Storm            → P50=116.5ms (+1392%), P99=239.8ms (synchronous DB writes drag down latency)
Phase 3: Storm + exhaustion   → P50=130ms, P99=9.4s (4/5 connections held, tail explodes)
Phase 4: Full recovery        → log-storm stop → P50 returns to < 10ms within 30s
Phase 5: PG HA Failover       → patroni primary stopped → patroni3 promoted within 5s ★
Phase 6: Redis Sentinel Failover ★ NEW
        docker stop redis-master → replica promoted in 6.1s → 0 errors / 865 requests
```

Measured latency timeline (2026-06-02, measured directly at FastAPI :8000)

Time →	T+0s	T+30s	T+75s	T+120s	T+180s(NEW)
/api/data					
P50(ms)	7.8	116.5	130.1	7.5	6.1
P99(ms)	32.1	239.8	9420	41.0	1030(spike)
	[Phase 1]	[Phase 2]	[Phase 3]	[Phase 4]	[Phase 6 Redis HA]
		log-storm	+conn-exhaust	recovery done	Sentinel switchover moment

P99=9420ms (Phase 3) = the tail wait after 4/5 of the pool is held, 0 errors (all eventually succeed)
Redis Failover P99=1030ms = redis-py auto-retry at the moment of Sentinel switchover, 0 errors ✓

[SLIDE 3-4] Diagnostic Thinking: the Observation Chain in Action

When you get the alert: "P99 Latency > 500ms"

Step 1: Look at the HTTP tier

```
http_request_duration_seconds P99 → confirm the abnormal endpoint is /api/data
```

Step 2: Look at the Cache tier

```
redis_cache_hits_total rate → Hit Rate is normal → Redis is fine
```

Step 3: Look at the DB tier

```
db_query_duration_seconds P99 → normal, slow queries are not the root cause  
db_active_connections_held → value is 4 → the connection pool is exhausted!
```

Step 4: Look at the Log Storm metrics

```
log_storm_active → 1  
log_writes_total rate → heavy writes  
log_write_duration_seconds P99 → 150ms per write
```

[SLIDE 3-4B] Diagnostic Conclusion (RCA)

Conclusion:

Each request adds a synchronous DB log write, which amplifies connection contention under high concurrency, ultimately exhausting the connection pool and causing overall latency to spike.

[SLIDE 3-6] Phase 5: PostgreSQL HA Failover (Patroni automatic leader election)

Failure simulation: sudden death of the Primary node

```
# Simulate: directly kill the Primary container (equivalent to a power cut)
docker stop api_monitoring-patroni2-1
```

Patroni automatic failover mechanism (etcd Raft DCS)

- Step 1: patroni2 stops updating the etcd Leader Lease (heartbeat interrupted)
- Step 2: etcd TTL(30s) counts down → the remaining Patroni nodes detect the leader is offline
- Step 3: the Replica with the most recent WAL (patroni3) wins the race → promoted to Primary
- Step 4: HAProxy re-routes automatically on the next check cycle (≈3s)
- Step 5: the application automatically reconnects to the new Primary → service restored

Measured results (tested 5/31, cluster health confirmed 6/2)

Metric	Value	Meaning
RTO (leader election time)	5 seconds	patroni3 promoted to new Primary within 5s
Errors during failover	9 / 5970	0.15% error rate
Service-interruption window	< 15 seconds	service restored after HAProxy re-routes automatically
Current cluster state (6/2)	patroni3=leader(timeline:3), patroni1/2=replica	operating normally
P50 after failover	7.8ms	back to baseline (Redis cache buffers DB pressure)

[SLIDE 3-7] Phase 6 (new): Redis Sentinel Failover Test ★

Failure simulation: the Redis master node suddenly stops

```
# Simulate: stop redis-master (equivalent to a node power cut)
docker stop api_monitoring-redis-master-1
# After Sentinel elects a new master, restart it (rejoins as a Replica)
docker start api_monitoring-redis-master-1
```

Sentinel automatic failover mechanism (quorum=2, down-after=5s)

```
Step 1: sentinel1/2/3 detect redis-master is not responding to PING (down-after-milliseconds=5000)
Step 2: Sentinel quorum=2 votes to confirm the master is down (SDOWN → ODOWN)
Step 3: a Sentinel Leader is elected (Raft-like mechanism) → initiates failover
Step 4: redis-replica1 is promoted to the new master
Step 5: the redis-py Sentinel client automatically calls SENTINEL get-master-addr-by-name
Step 6: the application transparently switches to the new master, the cache keeps serving
```

All 6 steps complete automatically; the application needs zero manual intervention and 0 lines of code changes.

[SLIDE 3-7] Phase 6 (cont.): Measured Results 2026-06-02

Measured data

Metric	Value	Meaning
Sentinel failover time	6.1 seconds	down-after=5s + election time
Total requests during failover	865	3 background workers, ~20s test window
Errors during failover	0	redis-py Sentinel client auto-retries 
P99 latency (failover moment)	1030ms	a brief retry spike at the switchover moment
P50 after failover	6.3ms	back to baseline
Final cluster state	replica1=new master; the old master auto-demotes to replica after restart	self-healing 

Key insight: Cache Miss as degradation protection

```
redis-master stops
  → during the Sentinel switchover, the cache is briefly unavailable (~6s)
  → cache miss → automatic fallback to PostgreSQL (graceful degradation)
  → 0 errors: users only see source="database" instead of source="cache"
```

Double protection: Redis Sentinel's fast recovery (6s) + Cache-Aside degradation (0 errors) keep the service

[SLIDE 3-5] Recovery Strategy: the Power of Feature Flags

Traditional approach vs modern approach

	Traditional approach	This design
Recovery method	Restart the service	<code>GET /api/anomaly/log-storm/stop</code>
Recovery time	Minutes (restart + healthcheck)	Instant (< 1 second)
Log retention	✗ logs may be lost after restart	✓ container not restarted, all logs fully retained
Root-cause traceability	Difficult	✓ the Grafana timeline is fully preserved

Key insight

"Restore the service first" and "preserve the scene" are not mutually exclusive.
A feature flag lets you disable the problematic feature without restarting, while keeping the full timeline of all metrics and logs intact, becoming irreplaceable evidence for post-incident analysis.

CHAPTER 4

Operational Insights & Future Defense

[SLIDE 4-1] Why This Log Storm Was Preventable

Three design flaws (common anti-patterns)

Flaw 1: Synchronous Write

The log write shares the same thread and DB connection as the main business flow
→ any log-write latency is added directly to the API response time

Flaw 2: No Pool Isolation

Log INSERTs and business SELECT/UPDATEs share the same connection pool
→ during a log spike, logs steal connections from business queries

Flaw 3: No Load Testing

The dev environment only had 1-2 RPS, never triggering pool exhaustion
→ the feature looked perfectly fine in test, then crashed immediately in production

[SLIDE 4-2] Defensive Architecture Design

Solution 1: Async Logging / Log Buffering

Now (synchronous):

API Request → business logic → [synchronous DB Log INSERT] → Response
 ↑ this causes latency

Improved (asynchronous):

```
API Request → business logic → Response
                ↓ non-blocking
```

Buffer Queue → Background Worker → DB INSERT

Log writes no longer block the main thread, so API response time is unaffected.

Solution 2: Connection-pool isolation

Business pool (small and fast)

```
business_engine = create_engine(db_url, pool_size=5)
```

```
# Log pool (can be rate-limited independently)
```

```
log_engine = create_engine(db_url, pool_size=2, max_overflow=0)
```

[SLIDE 4-2B] Defensive Architecture Design (cont.)

Solution 3: Dynamic log-level adjustment

Production logs only WARNING and above by default.

Temporarily raise to DEBUG only when debugging, then immediately lower it again.

Avoids "full-volume logging crushing the system."

Practical implementation suggestions

- Normally: mainly INFO/WARNING, to maintain stable throughput
- During an incident: briefly raise to DEBUG, bound to a trace_id
- After an incident: lower back to the default level, so the observability system does not crush the main system

[SLIDE 4-3] The Next Step in Observability: Completing the Three Pillars

Already in place:

- ✓ Metrics (Prometheus + Grafana)
- ✓ Logs (Docker container logs + the `api_logs` table)

Next to complete:

- Traces (distributed tracing)

What problem does tracing solve?

Current observation chain: metrics find "which tier has a problem"

Adding tracing: find "which request, in which service, took how long"

Practical approach:

Add a `correlation_id` (or `trace_id`) to each request

Record the same `trace_id` in the API logs

Associate the same `trace_id` with DB slow-query logs too

→ one ID strings together the complete evidence for the whole request chain

[SLIDE 4-4] Architecture Expansion Roadmap

Architecture evolution: from demo to production

Already implemented (this project):

- Docker Compose (18 containers)
- PostgreSQL HA (Patroni) ✓
- etcd 3-node Raft DCS ✓
- HAProxy automatic routing ✓
- WAL Streaming Replication ✓
- Feature Flag (API call) ✓
- Redis Sentinel HA ✓
- container logs
- prometheus scrape

Future (production hardening):

- Kubernetes (K8s)
- Patroni on K8s (Operator)
- etcd on K8s (managed)
- Envoy / NGINX Ingress
- Multi-region HA
- ConfigMap dynamic updates
- Redis Cluster (horizontal scaling)
- Fluentd → Elasticsearch → Kibana
- Prometheus Operator + AlertManager

Both HA paths are fully implemented and passed failover testing:

PostgreSQL Patroni (RTO=5s, 0.15% errors) + Redis Sentinel (RTO=6.1s, 0 errors)

One key principle will not change

"A system must degrade under failure, not crash under failure."

Cache down → degrade to the DB

Log system under pressure → lower the log level, don't block business

DB pool full → reject immediately, return 503, instead of waiting forever

CLOSING SLIDE

Summary

Core capability	Implementation
Service Selection	PostgreSQL HA Cluster + Redis Sentinel HA + Hybrid Storage
Environment Setup	Docker Compose, 18 containers + 1 host service , healthcheck dependency chain
Monitoring	Prometheus + Grafana, 16 panels, 5 rows
Anomaly Simulation	7 chaos endpoints + dual HA failover (Patroni RTO=5s + Sentinel RTO=6.1s)
Metric-Based Diagnosis	four-tier observation chain, locating the root cause from HTTP → Cache → DB → System
Resolution & Prevention	Feature Flag instant rollback + dual HA automatic failover (0 manual intervention)

CLOSING SLIDE (2/2)

Summary (cont.)

System Architecture	Mermaid architecture diagram + OOP 5-layer + HA cluster topology
Use Case	user_traffic_generator.py **6-phase** interactive scenario (incl. HA failover)
Operational Insights	RED Method + P99 + production-incident scenario analysis + **measured RTO=5s data**

Core message (in one sentence):

"I designed this system not to show how much technology I can use, but to reproduce a typical production-incident scenario in an observable way, and to prove that, with the right monitoring design, such problems can be detected and prevented early."

"Adding PostgreSQL HA brings this system closer to a real production environment — it not only detects problems but also recovers automatically when a database node suddenly dies (RTO=5s), with the entire process fully recorded by Prometheus + Grafana."

Test summary (full run on 2026-05-31)

Item	Value
Total requests	user_api 3,507 + main_app 2,463 = 5,970
Total errors	9 (all within the failover window)
Overall availability	99.85%
HA failover RTO	5 seconds
Final P50 / P99	16.2ms / 50.4ms (back to baseline)

Attachments:

- *System architecture Mermaid source (document/system_architecture.mmd)*
- *OOP Class Diagram (document/class_diagram.mmd)*
- *Cache-Aside Sequence (document/sequence_cache_aside.mmd)*
- *Chaos Sequence (document/sequence_chaos.mmd)*